Project Proposal - Draft

Commercial Title: SpeculatorGate
Academic Title: A Reproducible Promotion Gate for Speculative Decoding Draft Models
Student Name: Jason G Hajdics
Student Number: W20114849
Programme: Higher Diploma in Science in Computer Science
Project Type: Work-Based Project
Project Category: AI/ML Development, DevOps, Cloud Computing, CI/CD Pipeline

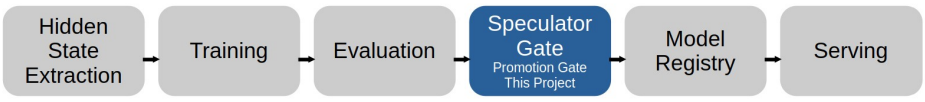
Preface

Many projects use Artificial Intelligence to get support while building software. This project builds tooling for AI systems themselves. The author is on work placement with a Red Hat team that develops open-source AI/ML model training and fine-tuning solutions, including pipelines for training and evaluating AI models.

Project Goal

This project aims to design, build, and evaluate a reproducible promotion gate for speculative decoding draft models. The system will support deployment decisions by providing statistical validation, token-level output consistency checks, workload robustness testing, and auditable decision lineage. SpeculatorGate consumes evaluation results and decides whether a drafter is fit for deployment. A candidate is promoted only when it beats baseline inference with statistically significant latency gains, while preserving output quality, and with every decision recording full lineage.

SpeculatorGate – Pipeline Architecture



Problem Statement

Running large language models is computationally expensive, largely because of the GPU capacity they consume. Speculative decoding reduces this cost by creating a small drafter model which guesses upcoming tokens while the large verifier model confirms or corrects them, producing identical output quality at potentially much higher speed. At present, however, deciding whether a trained drafter is actually good enough to deploy remains a manual, ad hoc process. Performance validation is orchestrated informally, with no reproducible record of which dataset revision, extraction configuration, or hyper-parameters produced a given drafter. Teams lack an automatic quality gate to prevent a

slow or misaligned drafter from reaching deployment, and on shared clusters, repeated extraction jobs can waste expensive GPU capacity.

Background

Existing tools solve fragments. Kubeflow, an open-source machine-learning workflow toolkit for Kubernetes, provides the SpeculativeDecodingTrainer which automates drafter training; MLflow tracks experiment runs; and Kubeflow Pipelines orchestrates steps. An evaluation component for speculative decoding reports speedup and acceptance metrics but it does not make deployment promotion decisions. The existing speculative decoding components of the Kubeflow Pipeline and Kubeflow Trainer were developed and published by the author's team. SpeculatorGate extends this existing tooling with a reproducible promotion framework based on statistical validation, output-equivalence checks, and workload robustness testing; it is planned to consume evaluation results and review of the evaluation component might be necessary during development.

At the time of writing, the author is not aware of an open-source workflow that integrates statistical validation, output-consistency checking, workload robustness testing, and auditable promotion decisions for speculative decoding draft models.

Stakeholders

The intended users of SpeculatorGate are ML engineers, MLOps engineers, and platform teams responsible for training, evaluating, and deploying speculative decoding draft models.

Proposed Technologies

Python, PyTorch, vLLM, Kubeflow SDK (Trainer, Pipelines), Red Hat OpenShift AI (Data Science Pipelines) as the primary hosted platform subject to availability, local Kubernetes (kind) for development and CI fall-back when cluster access is not provisioned, MLflow (tracking/registry), YAML manifests, Git/GitHub, pytest, Jira, Harvard Referencing via Zotero. Alternatives were considered, like Ray for orchestration but the Kubeflow SDK is currently the only SDK supporting speculative decoding training.

Data, Confidentiality and Ethics

All associated work is carried out in public, openly accessible repositories; no confidential, personal, or otherwise sensitive data will be used. Datasets are open-source benchmark sets. There are no anticipated GDPR, confidentiality, or other ethics compliance issues.

Methodology

Development will follow an Agile approach with six two-week sprints, weekly supervisor reviews, and progress tracked using Jira. Work will be delivered incrementally, beginning with the promotion-gate core, followed by pipeline integration, empirical evaluation, and final artefacts. The main project risk is the availability of a GPU-capable OpenShift cluster. Potential fall-backs include the Red Hat OpenShift AI developer trial, university cluster options, and a local kind cluster.